

L07: Sequence data & Recurrent Models

FICT, UTAR

Table of Contents

1. Sequence Data

- Tokenization
- Encoding

2. Recurrent Neural Network

- The simple RNN Cell
- Input-Output Configuration
- RNN Layer Configuration
- Signal Dimensions in RNN

Sequence Tasks

Speech Recognition



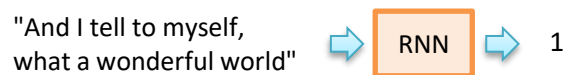
Machine translation



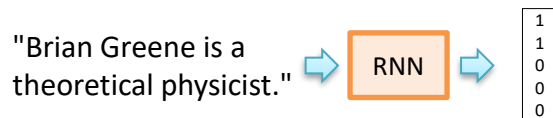
Music Generation



Sentiment Analysis



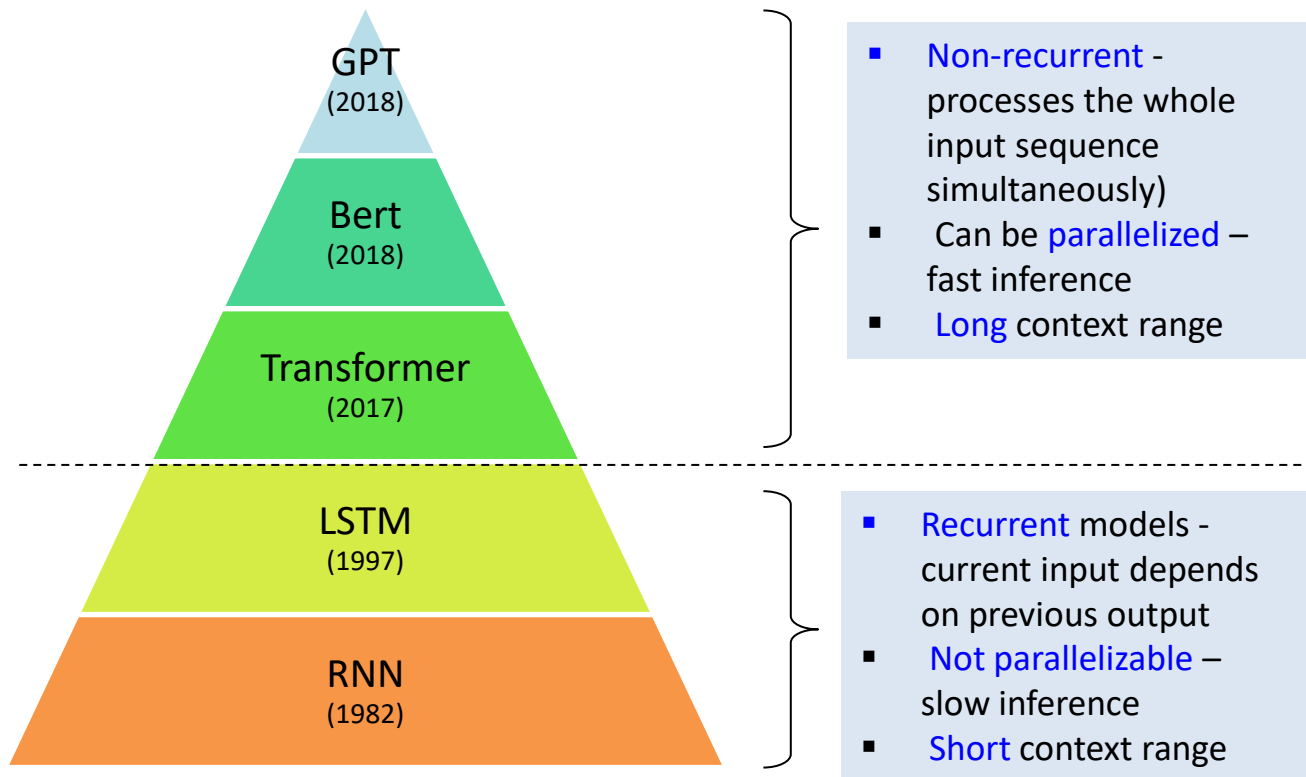
Named Entity Recognition



Video activity recognition



Sequence Models



Notation

- Named Entity Recognition (NER) task recognizes names in a sentence.

$\mathbf{X}^{(i)}$:	Ananda	Krishnan	and	Kah	Keat	passed	their	GCE	exam
	$\mathbf{x}^{(i)<1>}$	$\mathbf{x}^{(i)<2>}$	$\mathbf{x}^{(i)<3>}$	...	...	...	...	...	$\mathbf{x}^{(i)<9>}$
$\mathbf{Y}^{(i)}$:	1	1	0	1	1	0	0	1	0
	$\mathbf{y}^{(i)<1>}$	$\mathbf{y}^{(i)<2>}$	$\mathbf{y}^{(i)<3>}$	...	...	...	...	...	$\mathbf{y}^{(i)<9>}$

$$T_x^{(i)} = 9 \quad T_y^{(i)} = 9$$

- $\mathbf{X}^{(i)}$: The input sequence for the i^{th} sample
- $\mathbf{Y}^{(i)}$: The targeted output sequence for the i^{th} sample
- $T_x^{(i)}$: Length of input sequence for the i^{th} sample
- $T_y^{(i)}$: Length of targeted output sequence for the i^{th} sample
- $\mathbf{x}^{(i)<t>}$: The input feature at time step t for the i^{th} sample
- $\mathbf{y}^{(i)<t>}$: The targeted output at time step t for the i^{th} sample

Sequence Data Preprocessing Pipeline

Data Cleaning

- Remove noise (special characters, stop words, punctuation)
- Normalize text (lower-casing, stemming, lemmatization)

Tokenization

- Split text into words, subwords or characters
- Use techniques such as whitespace, BPE, WordPiece and TikToken

Batching

- Perform padding (inserting <pad> token) or truncation (removing tokens) ensures all sequence data have the same length to form a batch data.

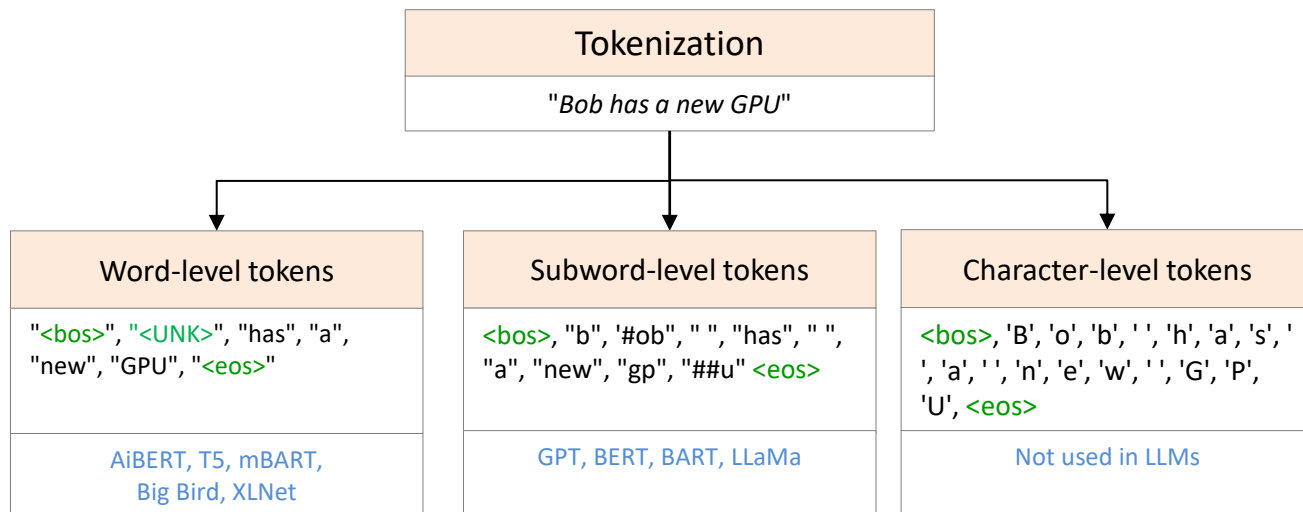
Encoding or Embedding

- Convert tokens into numerical form through encoding or embedding.
- Encoding: one-hot vector or one-hot integer representation
- Embedding: Word2Vec, GloVe, FastText, Contextualized Embedding Layer

Sequence Data Preprocessing: **Tokenization**

Tokenization

- Tokenization is the process of breaking text into smaller units called **tokens**, basic units of text processed by language models



- Special tokens: <UNK> unknown <PAD> padding
<BOS> Beginning of sentence <EOS> End of sentence

Comparison of Tokenization Methods

Word Tokenization	Subword Tokenization	Character Tokenization
Short sequences Easier to train	Not too long Easier to train	Long sequences Difficult to train
Large vocab	Moderately large vocab	Small vocab
Unable to handle out-of-vocab (OOV) words	Able to handle OOV words	Able to handle OOV words
Do not need training	Needs training	Do not need training

Classical BPE

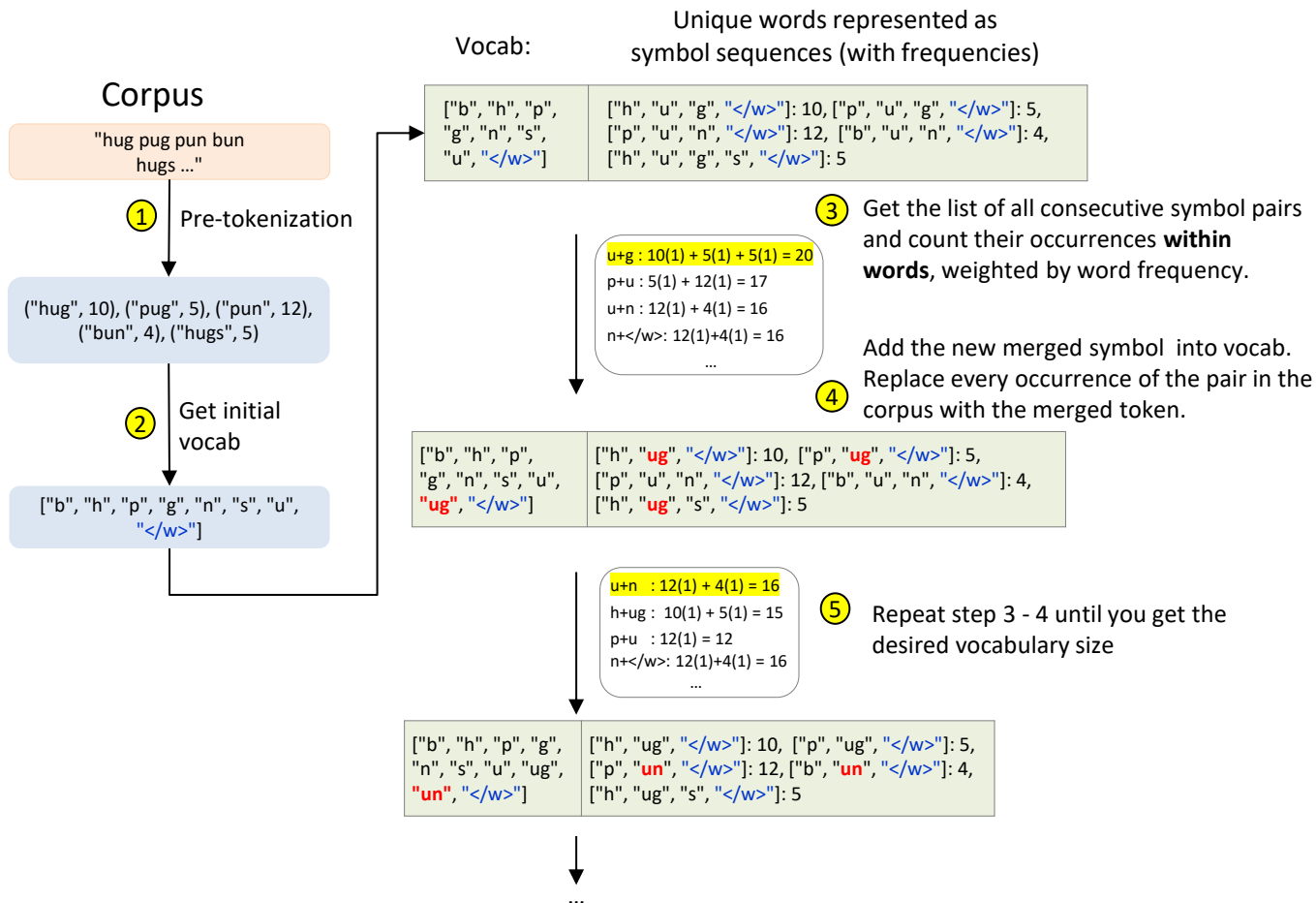
- A **subword** tokenization method that starts with characters (initial symbols) in the vocabulary. After pre-tokenizing text into words, it merges the most frequent adjacent symbol pairs **within each word** to build a fixed-size vocabulary.
- BPE stands for **Byte Pair Encoding**, originally used for data compression. Classical BPE tokenizer operates on characters rather than bytes (unlike byte-level BPE).
- Requires a **training** phase to learn merge rules from corpus statistics.
- During **inference**, classical BPE tokenizes text by greedily applying the learned merge rules in order within each word.
- Widely used in early NLP systems; modern systems use byte-level BPE or SentencePiece-based tokenizers.

Training Standard BPE

- 1. Pre-tokenization:**
Split text into words using whitespace or language-aware rules.
- 2. Initial vocabulary:**
Initialize the vocabulary with all characters and a special end-of-word marker `</w>`.
- 3. Count symbol pairs:**
Compute frequencies of adjacent symbol pairs within words.
- 4. Merge:**
Merge the most frequent symbol pair, add the new symbol to the vocabulary, and replace its occurrences in the corpus.
- 5. Repeat merge steps:**
Repeat steps 3 to 4 until the desired vocabulary size is reached.

[Reference: [huggingface/byte-pair-encoding](https://huggingface.co/byte-pair-encoding)]

Training Standard BPE (Example)



Tokenizing with Standard BPE

Inference:

- Pre-tokenize the text into words, then split each word into characters.
- Apply the merges within each word in the learned order (the order they are learned during training)
- Repeat the previous step until no more rules apply.

Byte-level BPE

- Standard BPE works on **characters**. Issues with character-level decoding:
 - Language-specific preprocessing
 - Issues with emojis, rare symbols, or mixed scripts
- Solution: **Byte-level BPE**
 - Similar to Standard BPE, except that it operates on raw UTF-8 **byte values** (0-255) instead of characters.
 - A utf-8 character can be represented as 1 to 4 bytes
 - "h" → [104] (UTF-encoding)
 - "é" → [195, 169] (UTF-encoding)
 - "😊" → [240, 159, 152, 138] (UTF-encoding)
 - Every text is first converted to bytes (rather than characters), then BPE merges frequent byte sequences.
- Used in models such as GPT-2, GPT-Neo, and RoBERTa

WordPiece

- A **subword** tokenization method similar to BPE (used in BERT, ALBERT).
- WordPiece uses a PMI-like scoring heuristic that favors pairs that co-occur more than expected (approximating likelihood improvement)."
- Merge Score:

$$\text{score} = \frac{\text{freq}(\text{pair})}{\text{freq}(\text{first}) \times \text{freq}(\text{second})}$$



Pointwise Mutual Information (PMI) measures how often two items occur together than expected by chance.

$$\text{PMI}(x, y) = \log \frac{P(x, y)}{P(x)P(y)}$$

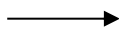
 **Example:**

- $p(\text{play}) = 0.1$, $p(\text{ing}) = 0.1$
- Expected if independent: $0.1 \times 0.1 = 0.01$
- Observed $P(\text{play}, \text{ing}) = 0.05$

Since $0.05 > 0.01 \rightarrow$ high PMI (good merge)

- Unlike BPE, which merges the most frequent pairs, WordPiece prefers pairs that are strongly associated."
- Uses prefix **##** to mark continuation subwords

"playing"



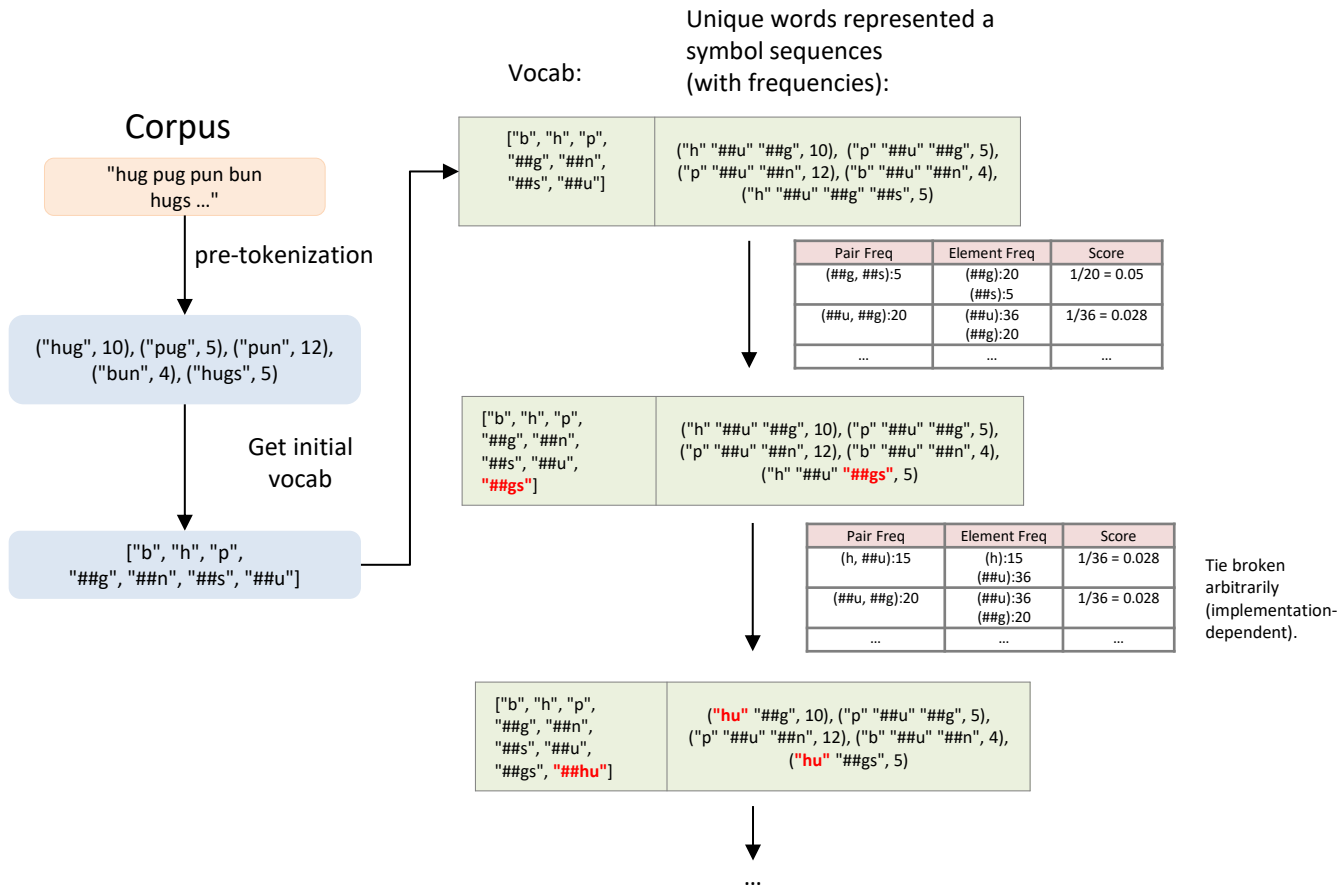
play

##ing

(both tokens exist in the vocabulary)

[Reference: [huggingface/wordpiece](https://huggingface.co/wordpiece)]

WordPiece: Example



SentencePiece

- Issue with **WordPiece**:

- Requires **pre-tokenization** into words.
- Some languages do not have explicit spaces between words.

Chinese	Japanese	English
我希望你在学习深度学习过程中过于快的时光。	ディープラーニングを急いで学ばないことを願っています。	I hope you have a great time learning deep learning.

- SentencePiece** treats raw text as a sequence of Unicode symbols and replaces whitespace as a special symbol (`_`) which marks a whitespace boundary.

Chinese:

我	希	望	你	在	学	习	深	度	学	习
---	---	---	---	---	---	---	---	---	---	---

 ...

English:

I		h	o	p	e		y	o	u		h
---	--	---	---	---	---	--	---	---	---	--	---

 ...

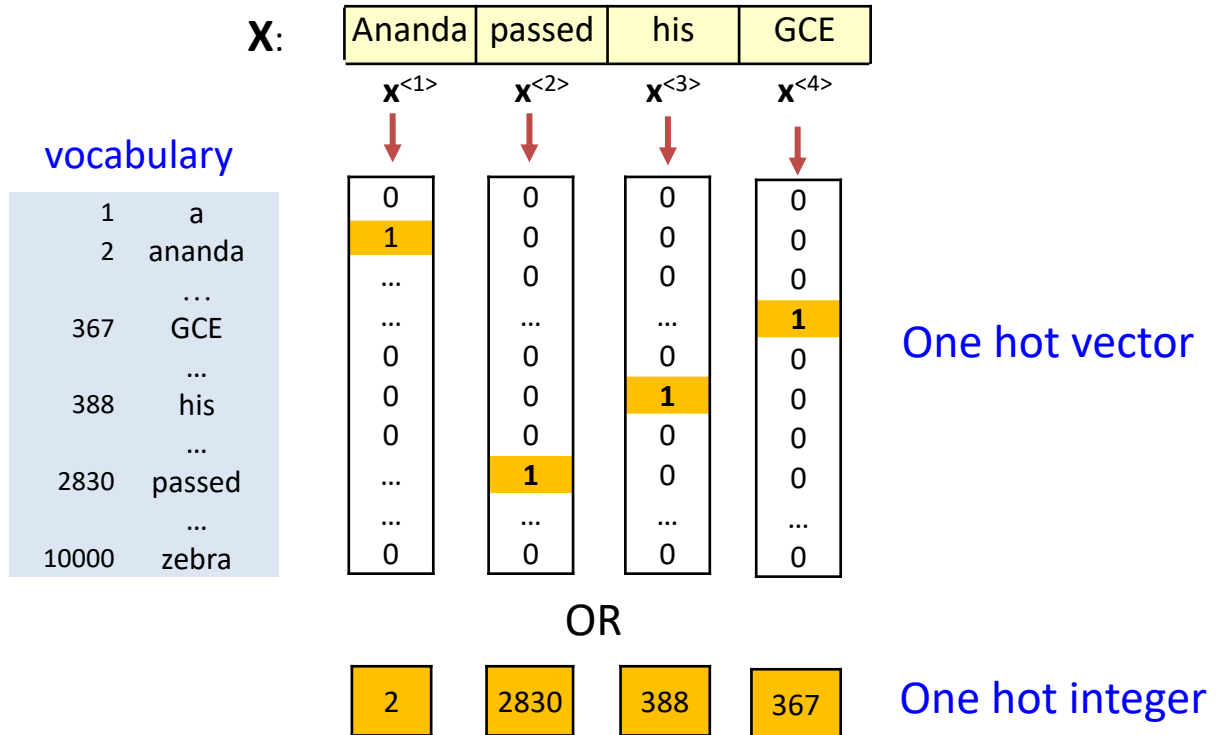
- SentencePiece is a framework, not a single algorithm – it can be trained using BPE or Unigram LM.
- No pre-tokenization required – SentencePiece works directly on raw text.

Reference: [huggingface/sentence-piece](https://huggingface.co/sentence-piece)

Sequence Data Preprocessing: **Encoding**

One-hot vector or One-hot integer representation

- Convert each token (string) into its vocabulary index (numerical values)



Weakness of One-hot Encoding

- High dimensional leads to **curse of dimensionality**
 - Easy to **overfit**
 - Every samples appear **equidistant** from all the other
- Treats all tokens as independent entities – **no notion of distance** between tokens.

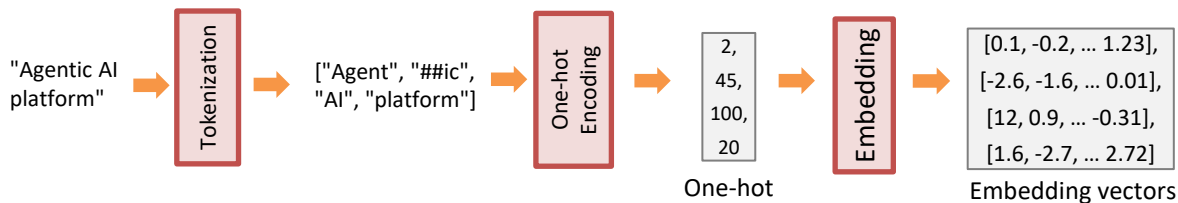
cat	1	[1 0 0 0 0 0 0 0 0 0]
tiger	2	[0 1 0 0 0 0 0 0 0 0]
computer	3	[0 0 1 0 0 0 0 0 0 0]

$$\text{sim}(\text{cat}, \text{tiger}) = \text{sim}(\text{cat}, \text{computer})$$

😬 But this does not make sense!

What is Token Embedding?

- The **embedding layer** converts **one-hot** representation into an **embedding vector**.



- Embedding vector is a **dense** 1D feature vector representing a particular token.
- Word embedding has more **semantic** meaning: the representations for **semantically similar** concepts are **nearer** to each other, and vice versa.

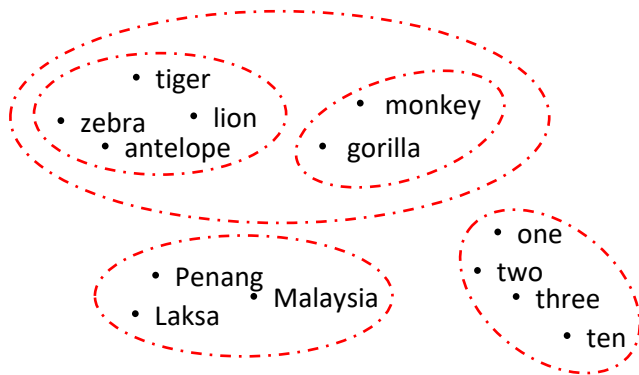
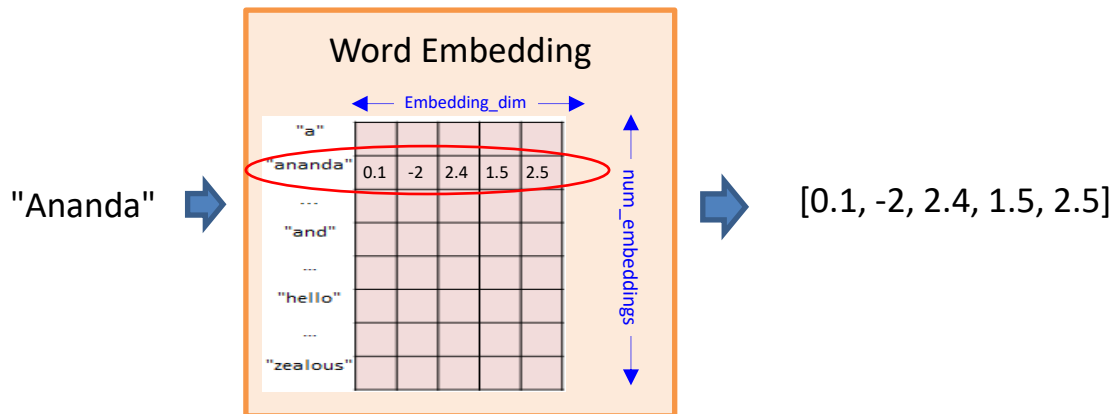


Figure: word embeddings projected into a 2-D space with t-SNE

The Embedding Layer

- The **embedding** layer is implemented using a **lookup table**. Given a word, it maps the word to the corresponding embedding vector.



- The lookup table is **learnt** through a training process.

torch.nn.Embedding

EMBEDDING

```
CLASS torch.nn.Embedding(num_embeddings, embedding_dim, padding_idx=None, max_norm=None,
                        norm_type=2.0, scale_grad_by_freq=False, sparse=False, _weight=None, _freeze=False,
                        device=None, dtype=None) [SOURCE]
```

A simple lookup table that stores embeddings of a fixed dictionary and size.

This module is often used to store word embeddings and retrieve them using indices. The input to the module is a list of indices, and the output is the corresponding word embeddings.

Parameters

- **num_embeddings** (*int*) – size of the dictionary of embeddings
- **embedding_dim** (*int*) – the size of each embedding vector
- **padding_idx** (*int, optional*) – If specified, the entries at `padding_idx` do not contribute to the gradient; therefore, the embedding vector at `padding_idx` is not updated during training, i.e. it remains as a fixed “pad”. For a newly constructed Embedding, the embedding vector at `padding_idx` will default to all zeros, but can be updated to another value to be used as the padding vector.
- **max_norm** (*float, optional*) – If given, each embedding vector with norm larger than `max_norm` is renormalized to have norm `max_norm`.
- **norm_type** (*float, optional*) – The p of the p -norm to compute for the `max_norm` option. Default 2.
- **scale_grad_by_freq** (*bool, optional*) – If given, this will scale gradients by the inverse of frequency of the words in the mini-batch. Default `False`.
- **sparse** (*bool, optional*) – If `True`, gradient w.r.t. weight matrix will be a sparse tensor. See Notes for more details regarding sparse gradients.

Variables

weight (*Tensor*) – the learnable weights of the module of shape $(\text{num_embeddings}, \text{embedding_dim})$ initialized from $\mathcal{N}(0, 1)$

Shape:

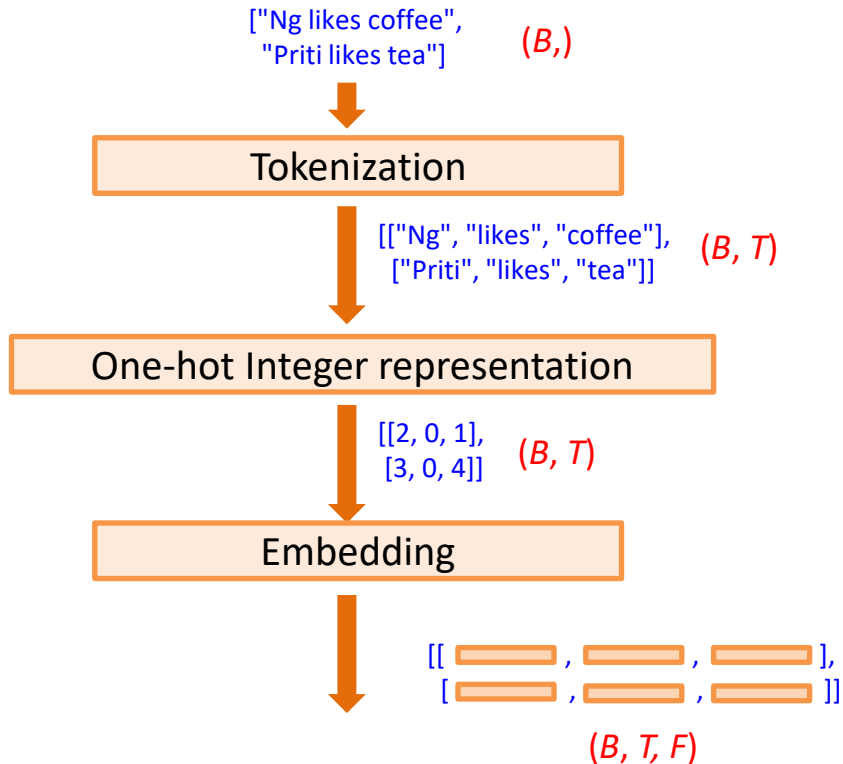
- Input: $(*)$, `IntTensor` or `LongTensor` of arbitrary shape containing the indices to extract
- Output: $(*, H)$, where $*$ is the input shape and $H = \text{embedding_dim}$

+

Examples:

```
>>> # an Embedding module containing 10 tensors of size 3
>>> embedding = nn.Embedding(10, 3)
>>> # a batch of 2 samples of 4 indices each
>>> input = torch.LongTensor([[1, 2, 4, 5], [4, 3, 2, 9]])
>>> embedding(input)
tensor([[-0.0251, -1.6902,  0.7172],
        [-0.6431,  0.0748,  0.6969],
        [ 1.4970,  1.3448, -0.9685],
        [-0.3677, -2.7265, -0.1685]],
       [[ 1.4970,  1.3448, -0.9685],
        [ 0.4362, -0.4004,  0.9400],
        [-0.6431,  0.0748,  0.6969],
        [ 0.9124, -2.3616,  1.1151]])
```

The Embedding Layer: Dimensions



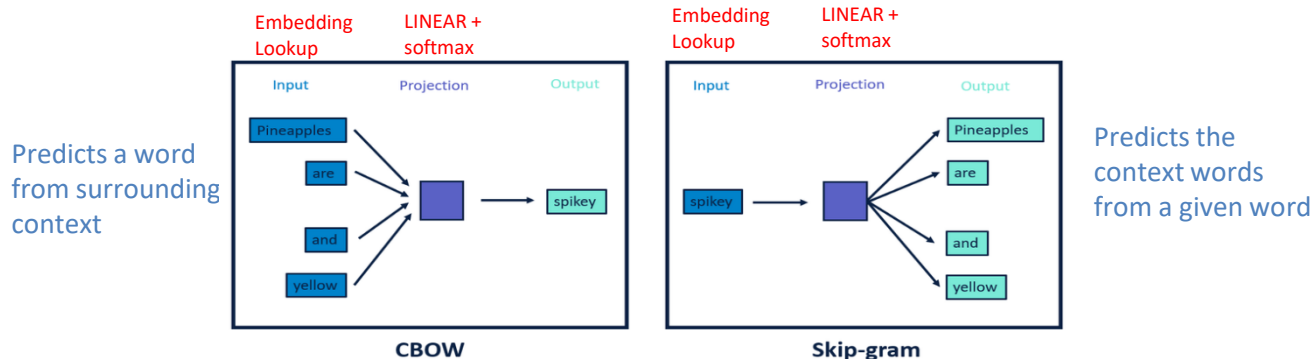
B : batch size

T : sequence length

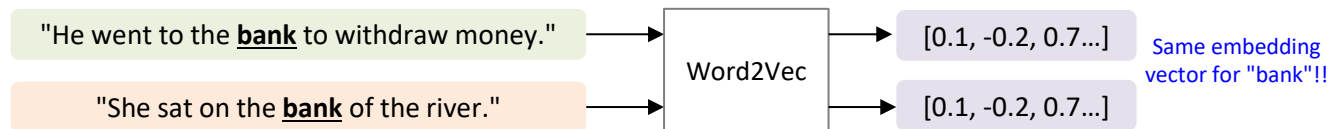
F : feature length = embedding dimension

Traditional Pre-trained Embedding Layer

- **Word2Vec**, **GloVe**, **FastText** are traditional pre-trained embedding layer.
- For example, the **word2Vec** can be trained in two different ways:



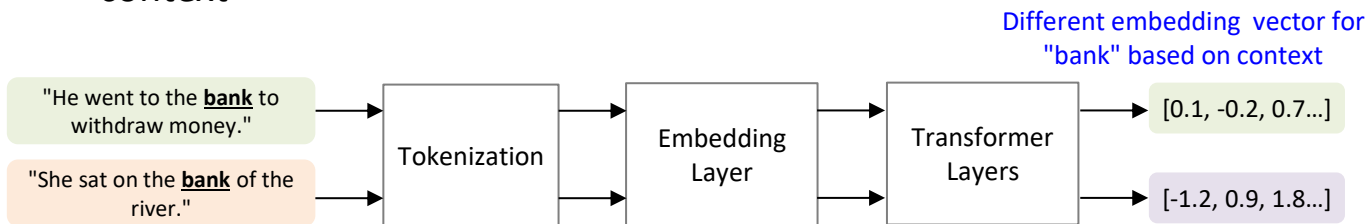
- Weaknesses:
 - **Lacks** sentence structure **understanding** (word order is ignored).
 - **Fixed** vector for each word: cannot handle **polysemy** (same embedding, different meanings) and cannot adapt to context.



- Largely replaced by **contextual embeddings** from Transformer-based models

Contextualized Embedding Layer

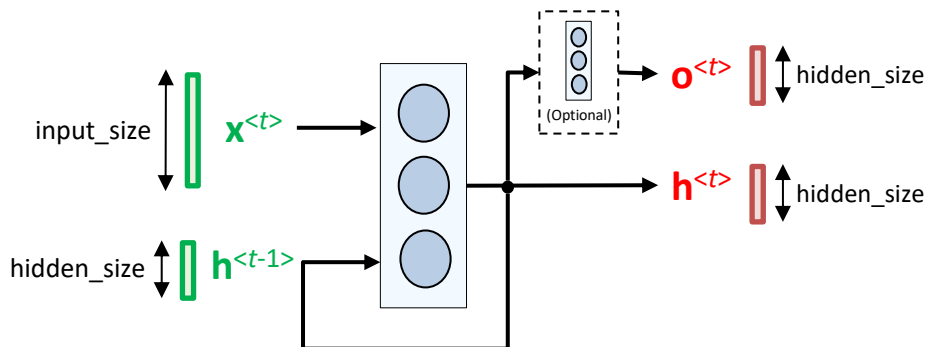
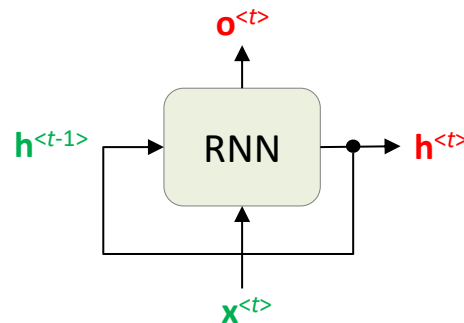
- Examples: BERT, RoBERTa, GPT
- Key Idea:
 - Word vectors should **change** depending on sentence **context**
 - Captures **polysemy** (different meanings of a word)
- Uses deep learning (transformers) to learn **context-aware embedding vectors**:
 - **Tokenization**: convert input text into tokens
 - **Embedding layer**: maps tokens to initial dense vector
 - **Transformer layers**: context is computed using self-attention and deep layers
- The same token can have **different** representation depending on the context



Recurrent Neural Network (RNN): **Simple RNN Cell**

Simple Recurrent Neural Network (RNN) Cell

- Given the *current input* ($\mathbf{x}^{<t>}$) and the *previous hidden state* ($\mathbf{h}^{<t-1>}$), a **simple RNN Cell** computes a *hidden state* ($\mathbf{h}^{<t>}$) and use it to generate the *output value* ($\mathbf{o}^{<t>}$)
- A simple RNN cell is a **fully connected layer** with **feedback** from previous time steps. The output is generated from the hidden state. For this lecture, we shall assume $\mathbf{o}^{<t>} = \mathbf{h}^{<t>}$.



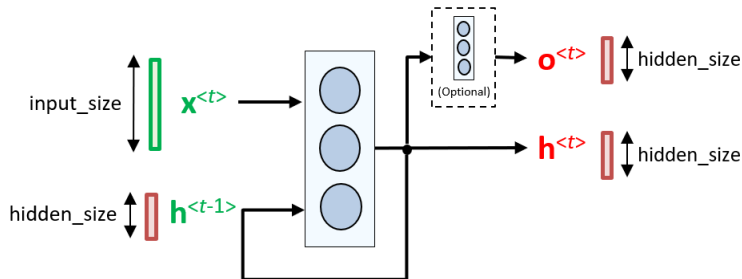
hidden size

= #neurons in the layer
= length of current hidden state
= length of output vector
= length of previous hidden state input

input size

= length of the input vector

Forward Propagation



- Two phases: **Linear** $\rightarrow$ **activation** (tanh / relu)
- Vectorized formulation for a batch data:

$$\mathbf{O}^{<t>} = \mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>} \mathbf{W}_{hh} + \mathbf{X}^{<t>} \mathbf{W}_{hx} + \mathbf{b}^T)$$

The diagram illustrates the vectorized formulation of the RNN forward pass. It shows the following components and their dimensions:

- $\mathbf{H}^{<t-1>}$ (orange stacked rectangles): $(B, \text{hidden_size})$
- $\mathbf{W}_{hh}$ (red square): $(\text{hidden_size}, \text{hidden_size})$
- $\mathbf{X}^{<t>}$ (green stacked rectangles): $(B, \text{input_size})$
- $\mathbf{W}_{hx}$ (green rectangle): $(\text{input_size}, \text{hidden_size})$
- $\mathbf{b}^T$ (blue horizontal bar): $(1, \text{hidden_size})$

Blue dashed arrows indicate the flow of data and the resulting dimensions of the intermediate sums.

Example

Given hidden size = 5, input size = 100, batch size = B

$$\mathbf{O}^{<t>} = \mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>} \mathbf{W}_{hh} + \mathbf{X}^{<t>} \mathbf{W}_{hx} + \mathbf{b}^T)$$

Diagram illustrating the dimensions of the tensors in the equation:

- $\mathbf{O}^{<t>}$ and $\mathbf{H}^{<t>}$ have dimensions $(B, 5)$.
- $\mathbf{H}^{<t-1>}$ has dimensions $(B, 5)$.
- $\mathbf{W}_{hh}$ has dimensions $(5, 5)$.
- $\mathbf{X}^{<t>}$ has dimensions $(B, 100)$.
- $\mathbf{W}_{hx}$ has dimensions $(100, 5)$.
- $\mathbf{b}^T$ has dimensions $(1, 5)$.

RNN Operations (t = 1)

- During inference, use a *for* loop to compute the output at each time step t given the current input and the previous hidden state

Time step = 1

Ground Truth $\mathbf{y}$

1	0	0	1	0
---	---	---	---	---

Predicted output $\hat{\mathbf{y}}$

0.9				
-----	--	--	--	--

Current output $\hat{y}^{<1>}$

Set initial previous hidden state to 0

0

$\mathbf{h}^{<0>}$

FC

FC layer transforms the RNN feature to class scores.

$\mathbf{o}^{<1>}$

RNN

$\mathbf{h}^{<1>}$

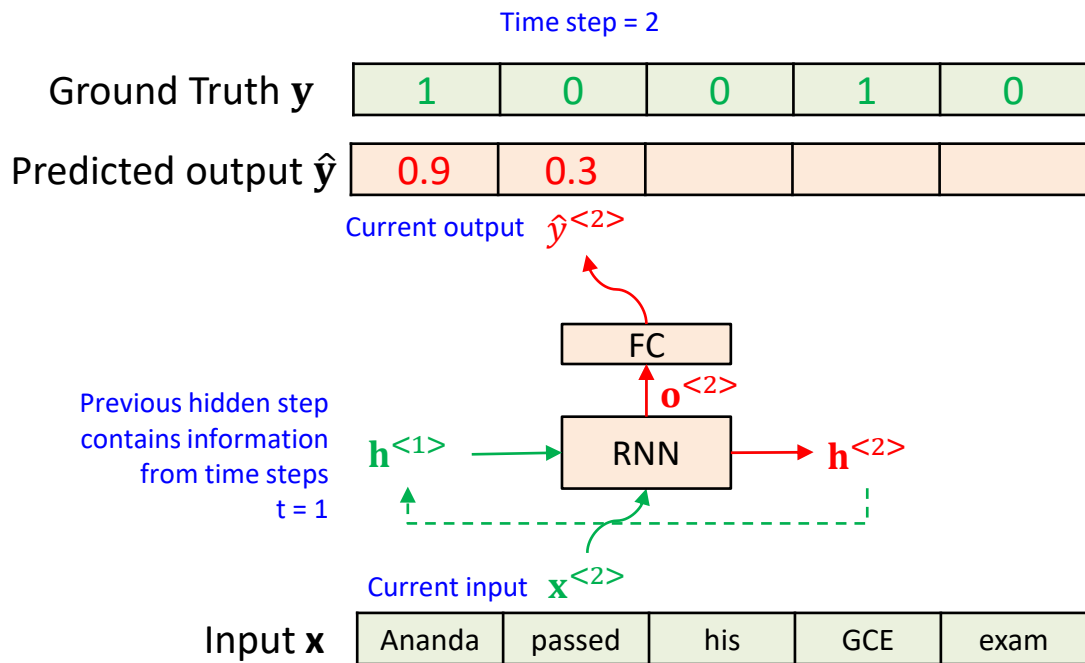
Current input $\mathbf{x}^{<1>}$

Input $\mathbf{x}$

Ananda	passed	his	GCE	exam
--------	--------	-----	-----	------

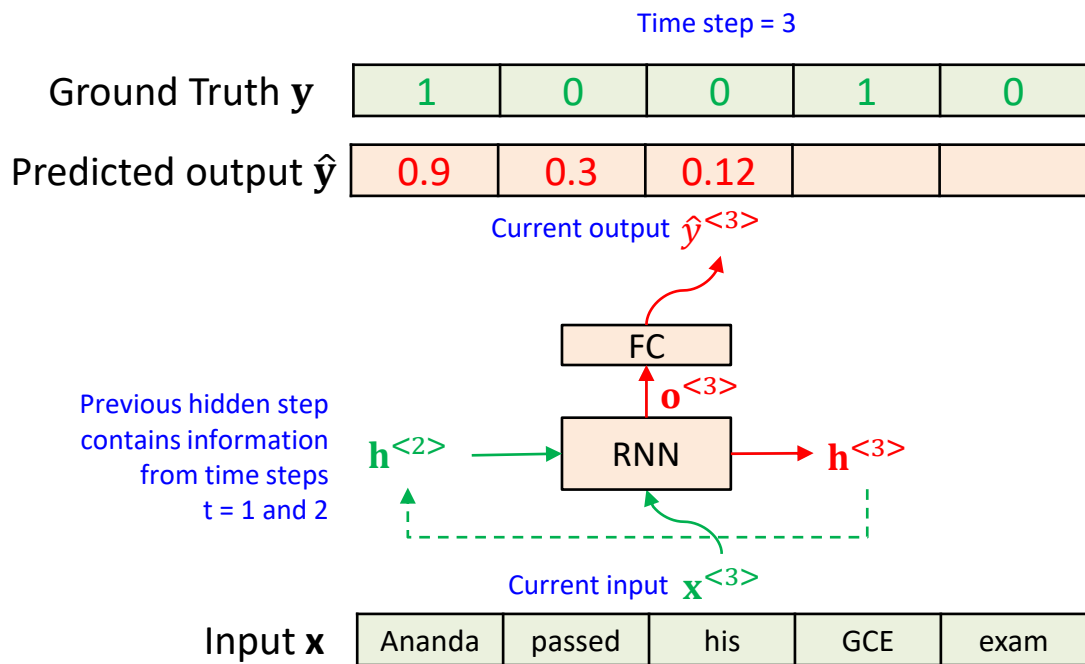
RNN Operations ($t = 2$)

- During inference, use a *for* loop to compute the output at each time step t given the current input and the previous hidden state



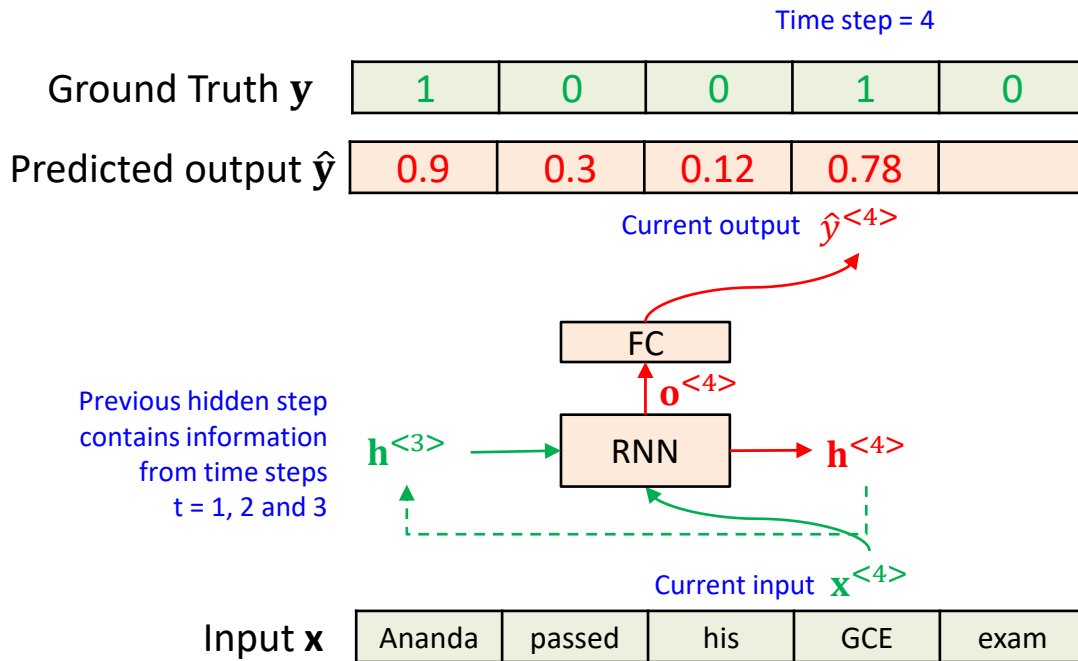
RNN Operations (t = 3)

- During inference, use a *for* loop to compute the output at each time step t given the current input and the previous hidden state



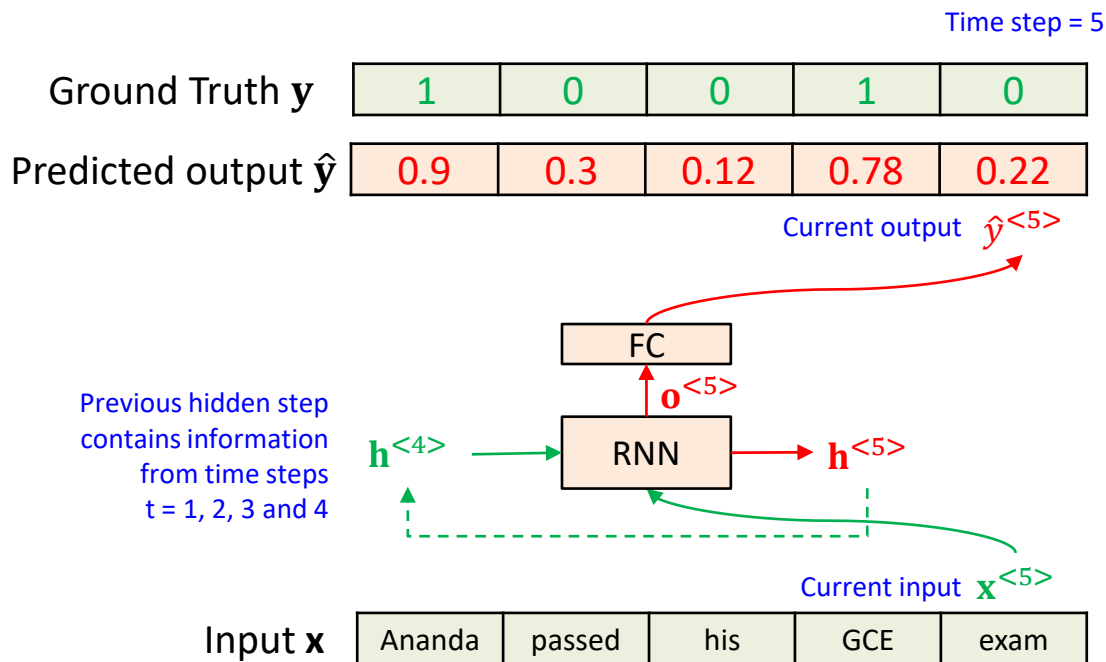
RNN Operations (t = 4)

- During inference, use a *for* loop to compute the output at each time step t given the current input and the previous hidden state



RNN Operations (t = 5)

- During inference, use a *for* loop to compute the output at each time step t given the current input and the previous hidden state



RNN

CLASS `torch.nn.RNN(*args, **kwargs)` [\[SOURCE\]](#)

Applies a multi-layer Elman RNN with `tanh` or `ReLU` non-linearity to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

where h_t is the hidden state at time t , x_t is the input at time t , and h_{t-1} is the hidden state of the previous layer at time $t-1$ or the initial hidden state at time 0. If `nonlinearity` is `'relu'`, then `ReLU` is used instead of `tanh`.

Parameters

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a stacked RNN, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1
- **nonlinearity** – The non-linearity to use. Can be either `'tanh'` or `'relu'`. Default: `'tanh'`
- **bias** – If `False`, then the layer does not use bias weights b_{ih} and b_{hh} . Default: `True`
- **batch_first** – If `True`, then the input and output tensors are provided as `(batch, seq, feature)` instead of `(seq, batch, feature)`. Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: `False`
- **dropout** – If non-zero, introduces a *Dropout* layer on the outputs of each RNN layer except the last layer, with dropout probability equal to `dropout`. Default: 0
- **bidirectional** – If `True`, becomes a bidirectional RNN. Default: `False`

Inputs: input, h_0

- **input** tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.
- **h_0** tensor of shape $(D * \text{num_layers}, H_{out})$ for unbatched input or $(D * \text{num_layers}, N, H_{out})$ containing the initial hidden state for the input sequence batch. Defaults to zeros if not provided.

where:

N = batch size
 L = sequence length
 $D = 2$ if `bidirectional=True` otherwise 1
 H_{in} = input_size
 H_{out} = hidden_size

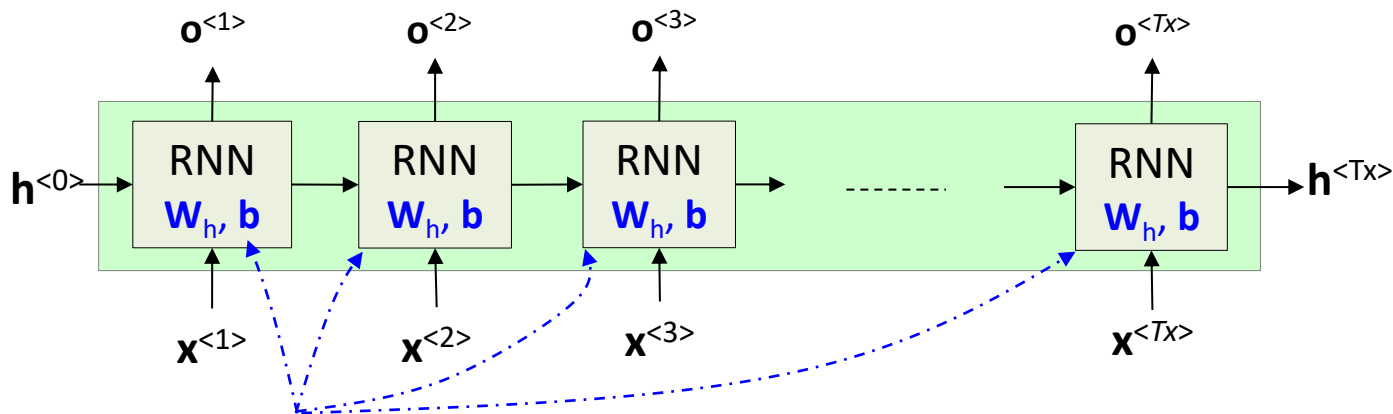
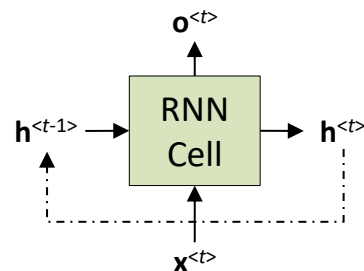
Outputs: output, h_n

- **output** tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (h_t) from the last layer of the RNN, for each t . If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence.
- **h_n** tensor of shape $(D * \text{num_layers}, H_{out})$ for unbatched input or $(D * \text{num_layers}, N, H_{out})$ containing the final hidden state for each element in the batch.

Recurrent Neural Network (RNN): **Input-Output Configuration**

Unrolling the Network

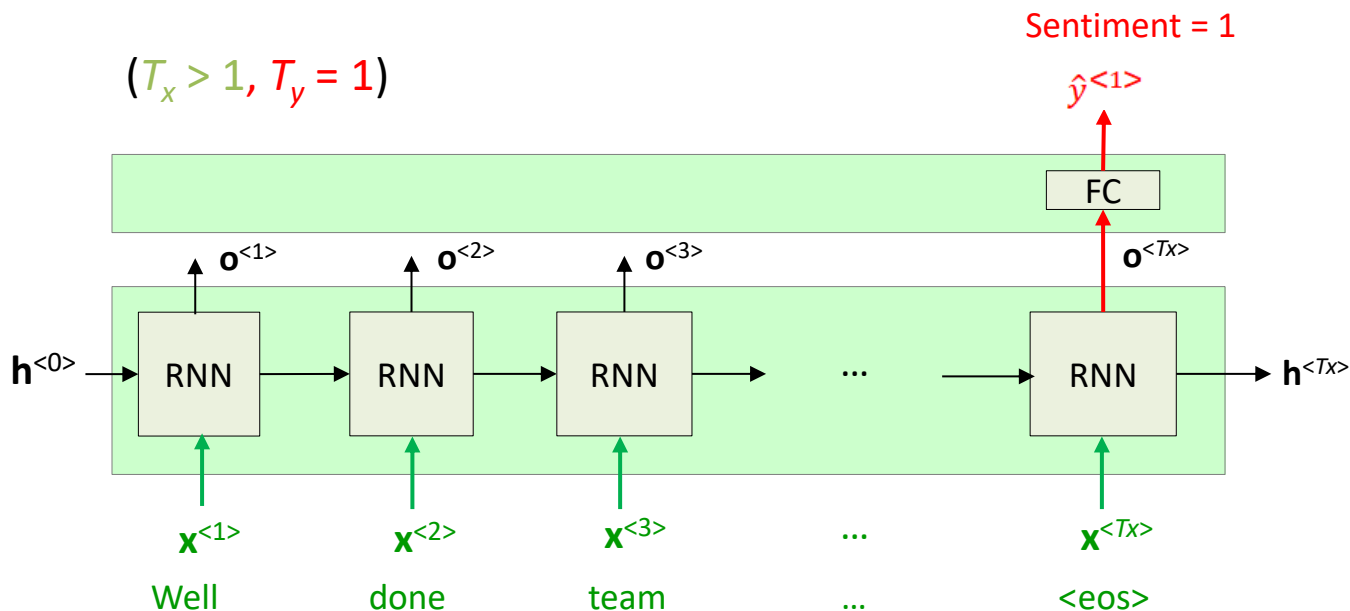
- During **training**, the feedback path prevents backpropagation!
- Solution: **unroll** the network to remove the feedback path.
- Depth of the network changes from batch to batch depending on the sequence length.



All the parameters are tied across all time step

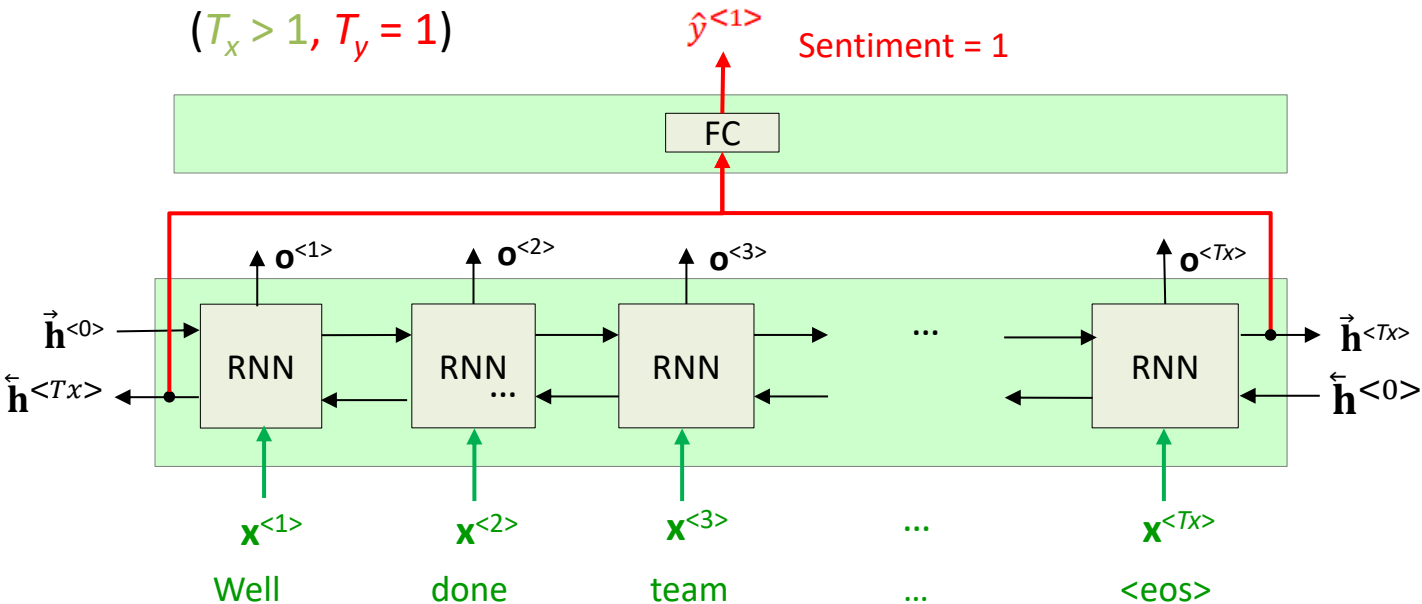
Many-to-One Network (unidirectional)

- Perform prediction at the last time step given an input
- Example: sentiment analysis (x: text, y: 0/1) for unidirectional model



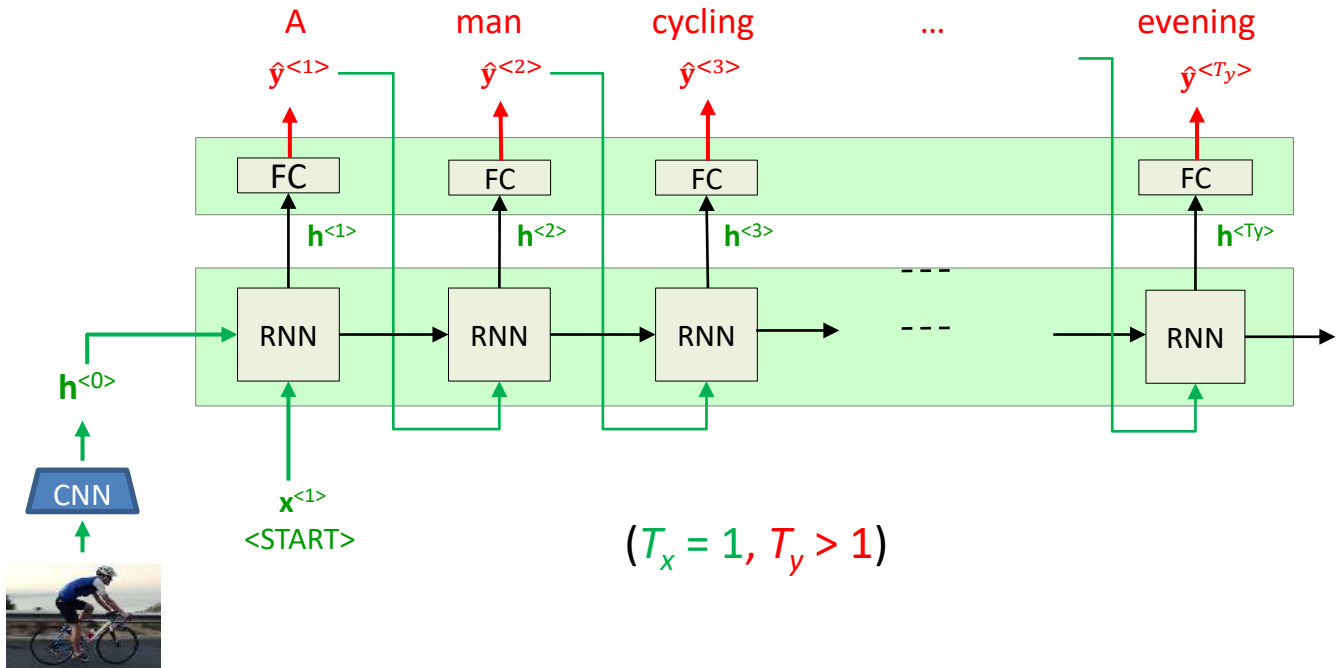
Many-to-One Network (bidirectional)

- Perform prediction at the last time step given an input
- Example: sentiment analysis (x: text, y: 0/1) for a bidirectional model



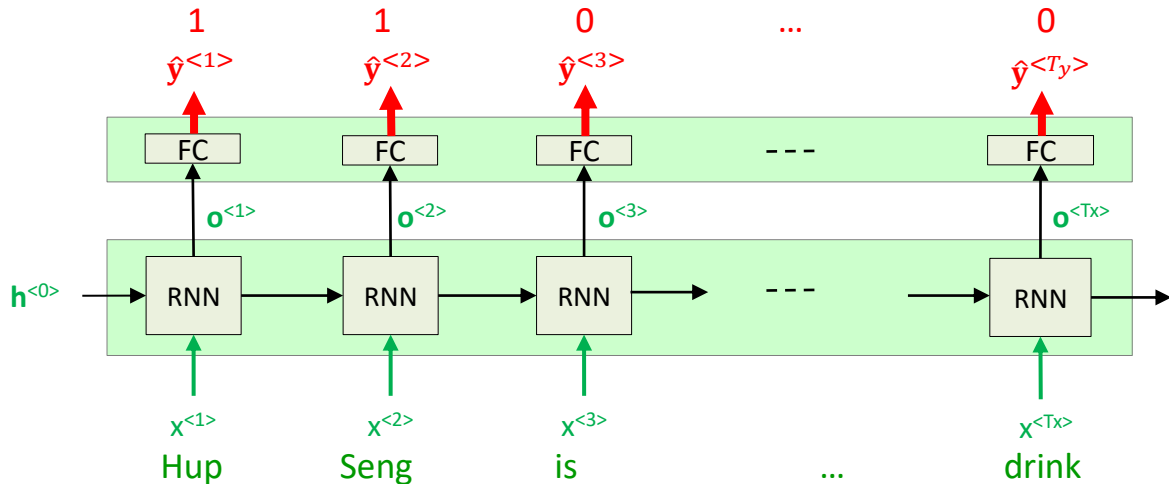
One-to-Many Network

- Generate an output sequence given non-sequence input
- Example: Image captioning (x = image, y = text), music generation ($x = \emptyset$, y = music keys)



Many-to-Many Network

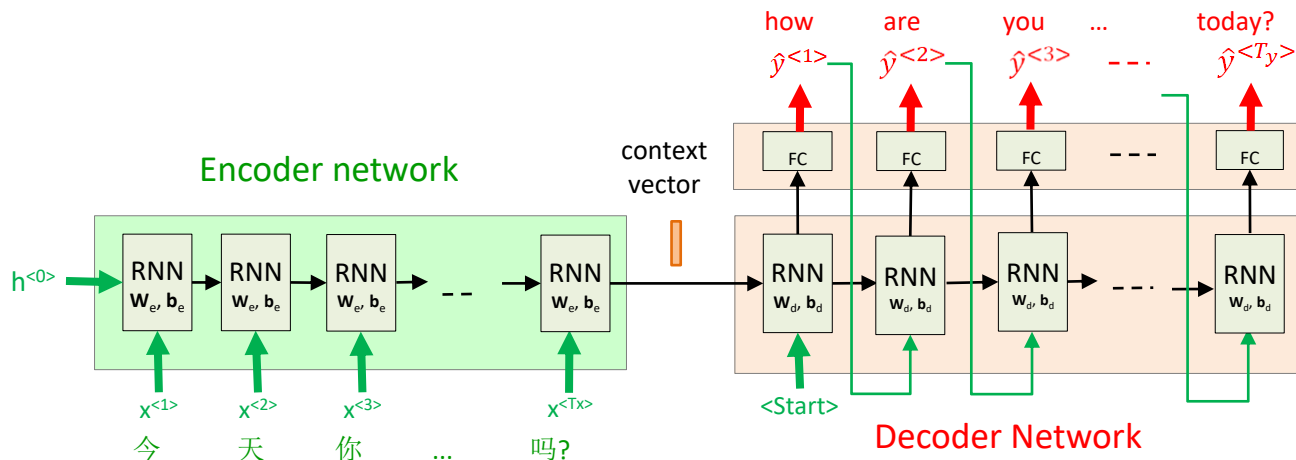
- Output sequence is aligned to the input sequence
- Example: Name entity detection, frame-level video annotation



$$(T_x = T_y)$$

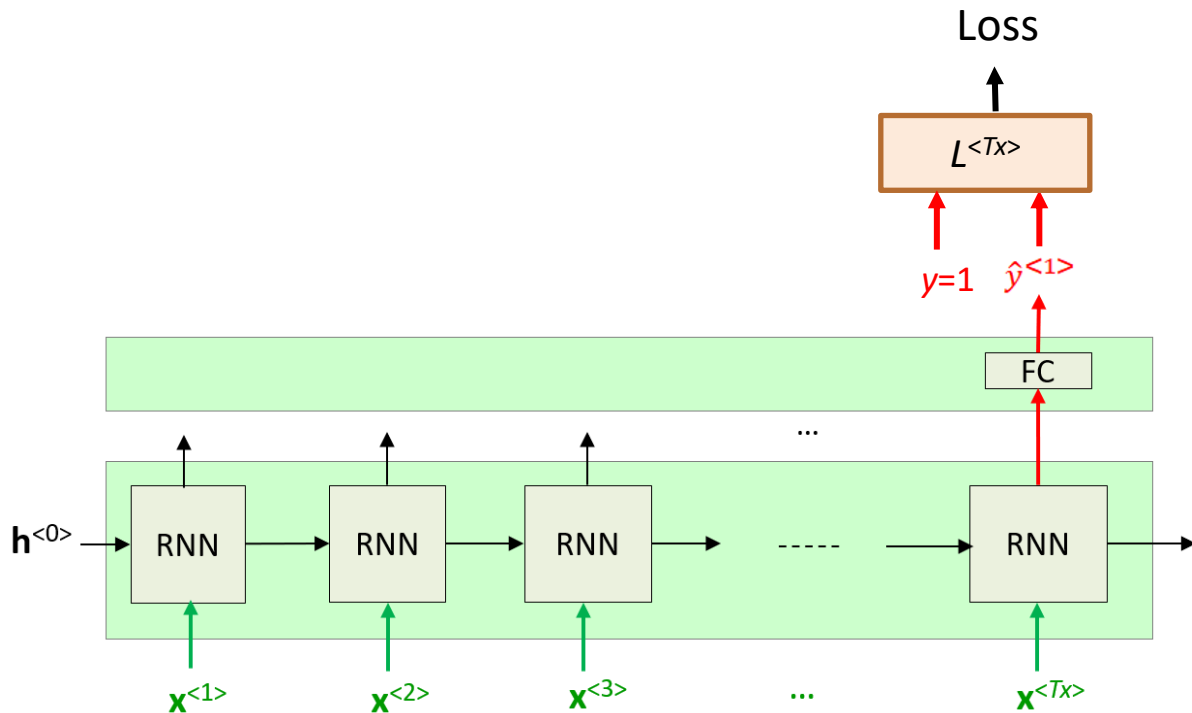
Encoder Decoder Network

- Example: Machine translation
- Output sequence is *not* aligned to the input sequence.
- The **encoder** generates the **context vector** to summarize the input
- The **decoder** generates the output sequence based on the context vector
- Different parameters for the encoder and decoder



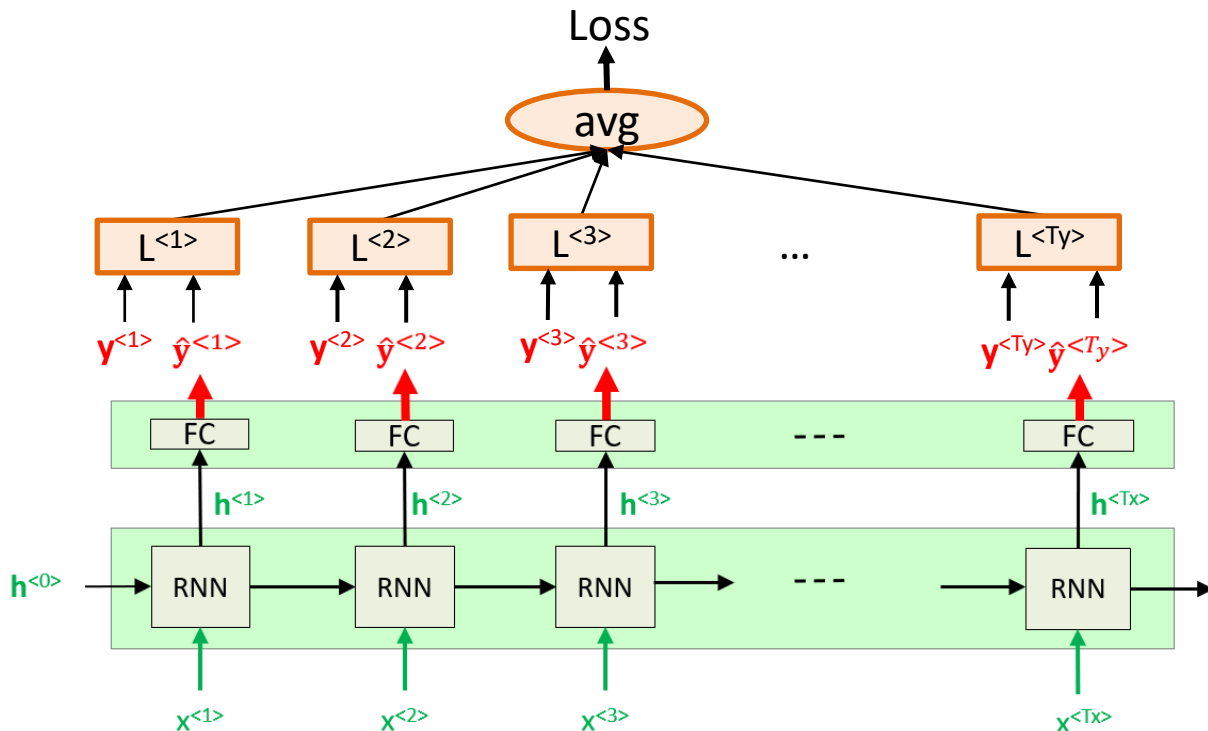
Loss Function (Scalar Output)

- For network with single output, simply apply the selected loss function (e.g., BCE)



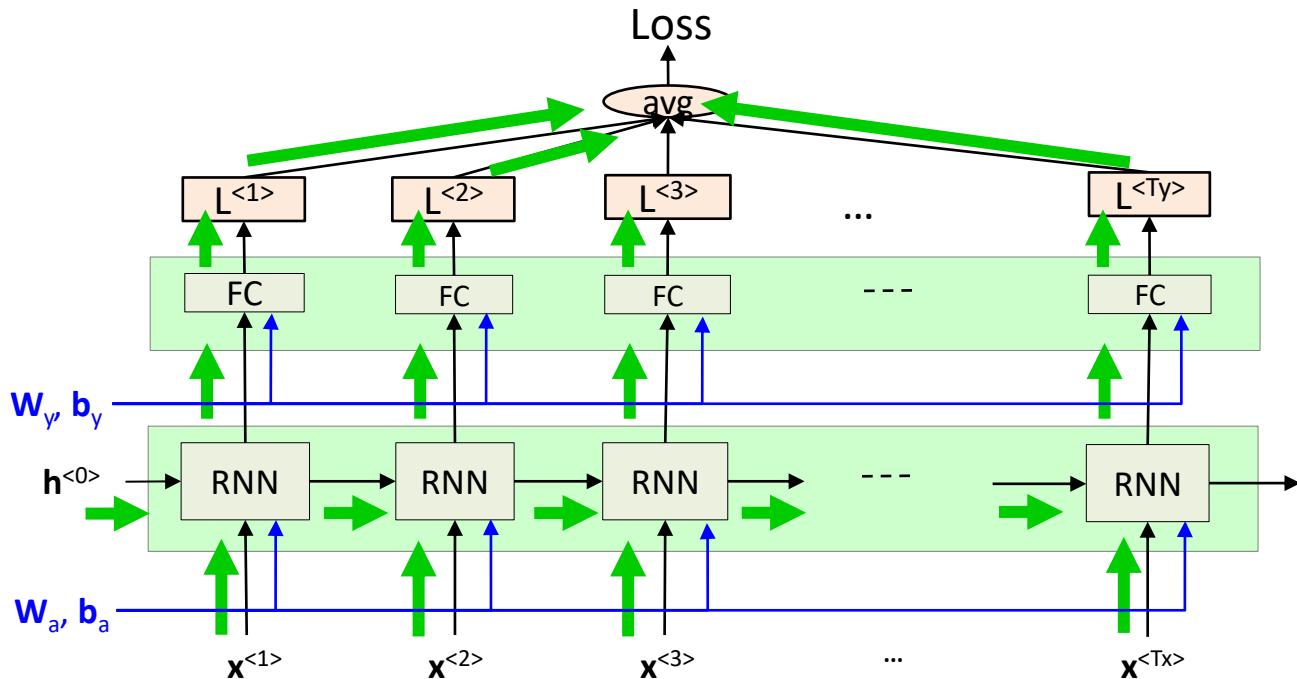
Loss Function (Temporal Output)

- For network with multiple output, compute individual losses at all time step and then average them



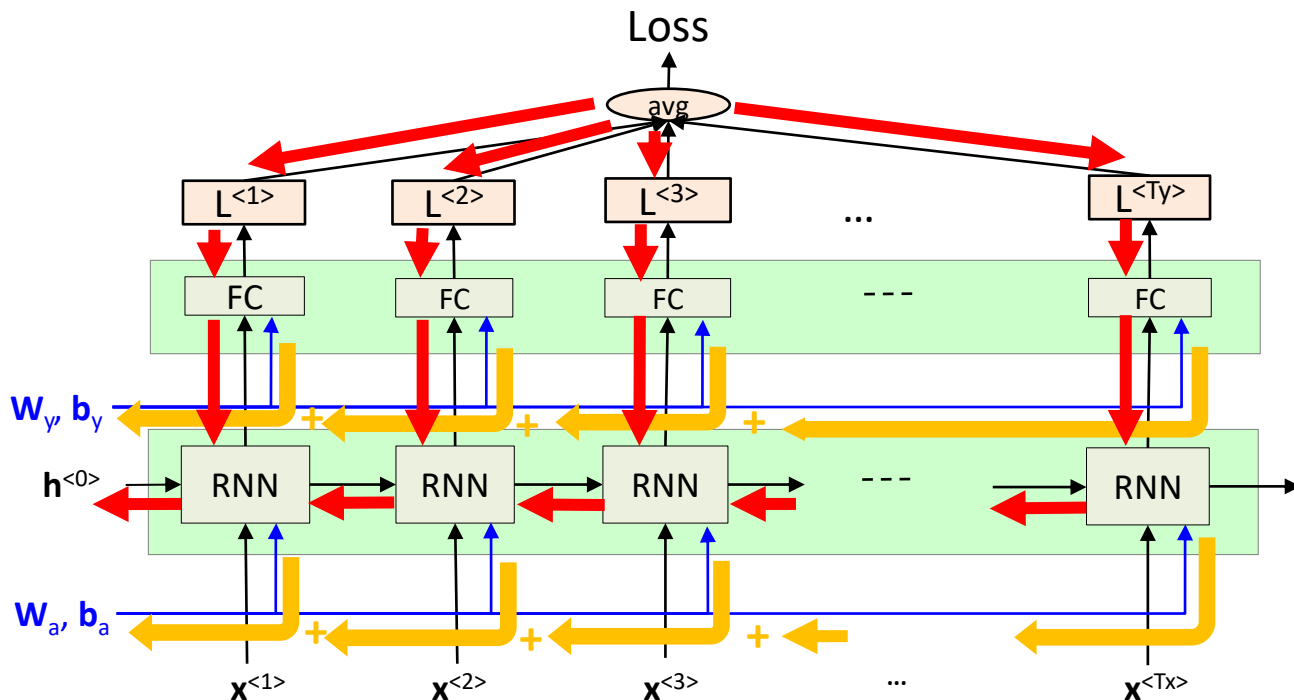
Forward propagation

- Forward from first to last time step.
- For output sequence, compute the individual loss at every time step and combine their losses at the end



Backpropagation Through Time (BPTT)

- There are many pathways to the parameters.
- The final gradients are computed by summing the gradients from different time stamps.



Sequence length of batch samples

- The **input sequences in a batch** must have the **same length** so that we can package them into one tensor and parallelize the computation
- The **input sequence of different batches** may have **different length** because they are processed at different iterations

Batch 1, $\mathbf{X}^{\{1\}}$:



Batch 2, $\mathbf{X}^{\{2\}}$:



Batch 3, $\mathbf{X}^{\{3\}}$:



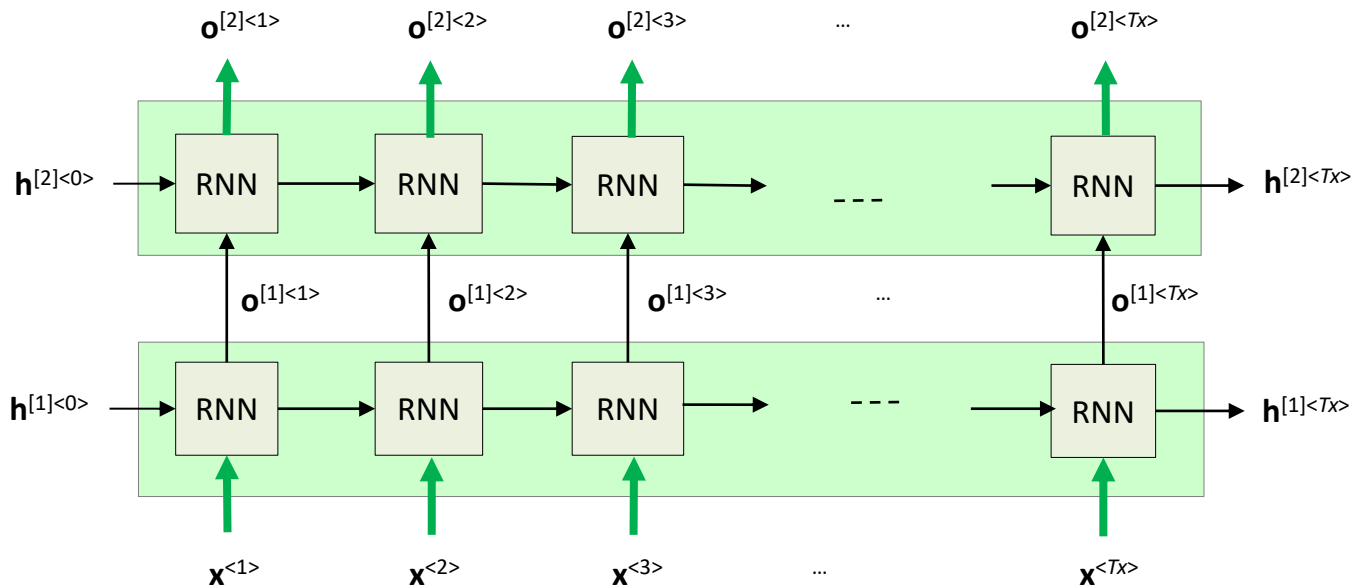
Batch 4, $\mathbf{X}^{\{4\}}$:



Recurrent Neural Network (RNN): Layer Configuration

Multi-layer RNN

- Stacking **multiple RNN layers** allows the network to learn higher level features



Bidirectional RNN (Bi-RNN)

- Inferencing in the forward direction is sometimes limited

Giving up / in / to / him / ... "?????????????"

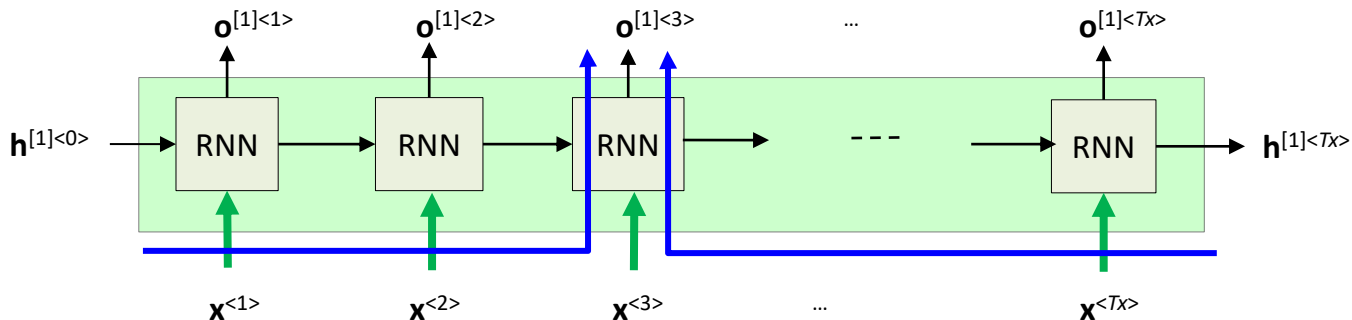
Too many possible answer just from 'Giving' alone

- Inferencing in the backward direction may be useful too

Giving up is unacceptable."

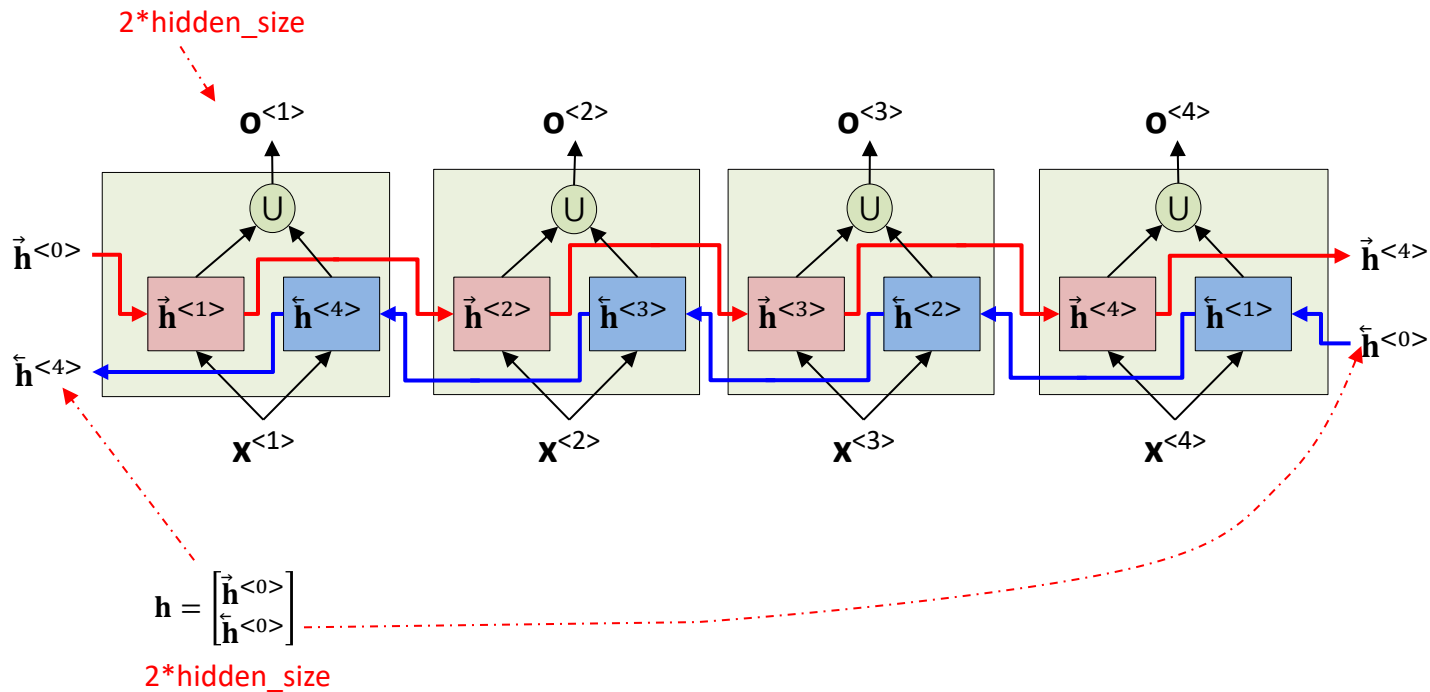
The latter part of the sentence can provide more context to narrow down the possible answer

- Bi-directional RNN performs inference in both direction



Bidirectional RNN (Bi-RNN)

- Perform prediction using information from **both** direction



Recurrent Neural Network (RNN): **Signal Dimensions in RNN**

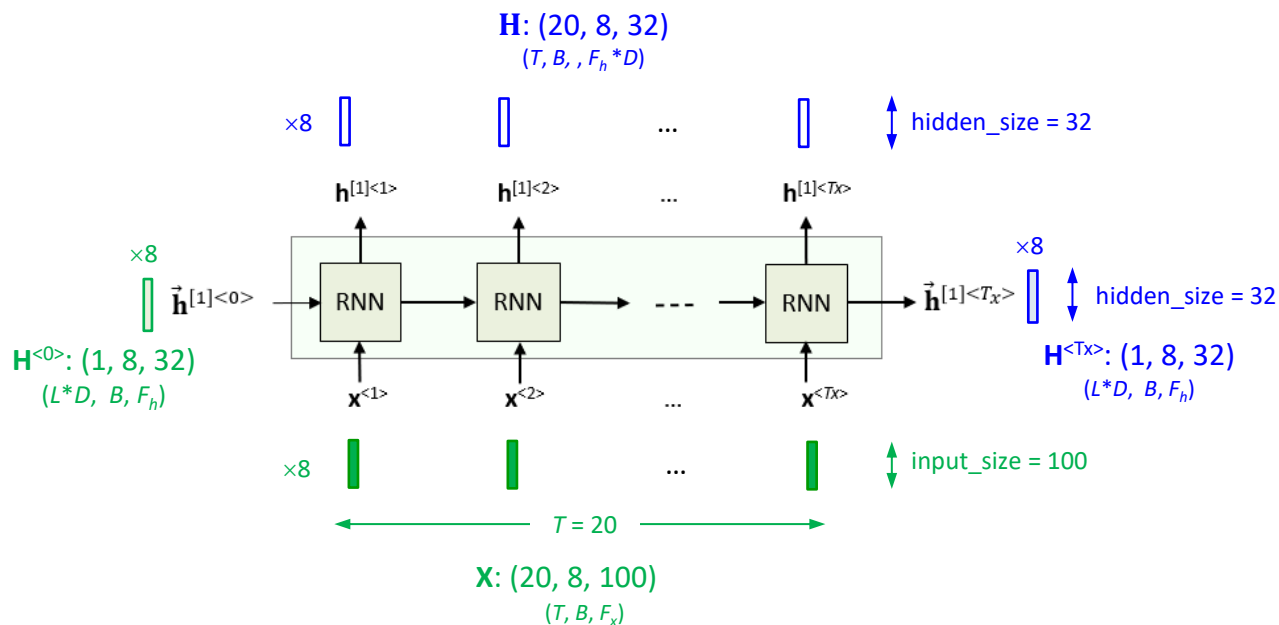
Data Format (Batch first or Temporal first)

<i>batch_first</i> = True	<i>batch_first</i> = False
• Data format: (B, T, F) • Shape of output: $(B, T, hidden_size)$ • Shape of input: $(B, T, input_size)$	• Data format: (T, B, F) • Shape of output: $(T, B, hidden_size)$ • Shape of input: $(T, B, input_size)$
Given batch size = 32, input_size = 100, hidden_size = 5, seq length = 15	
Shape of H: (32, 15, 5) ↑ RNN ↑ Shape of X: (32, 15, 100)	Shape of H: (15, 32, 5) ↑ RNN ↑ Shape of X: (15, 32, 100)

Signal Dimensions in RNN

- Batch size $B = 8$, Sequence length $T = 20$

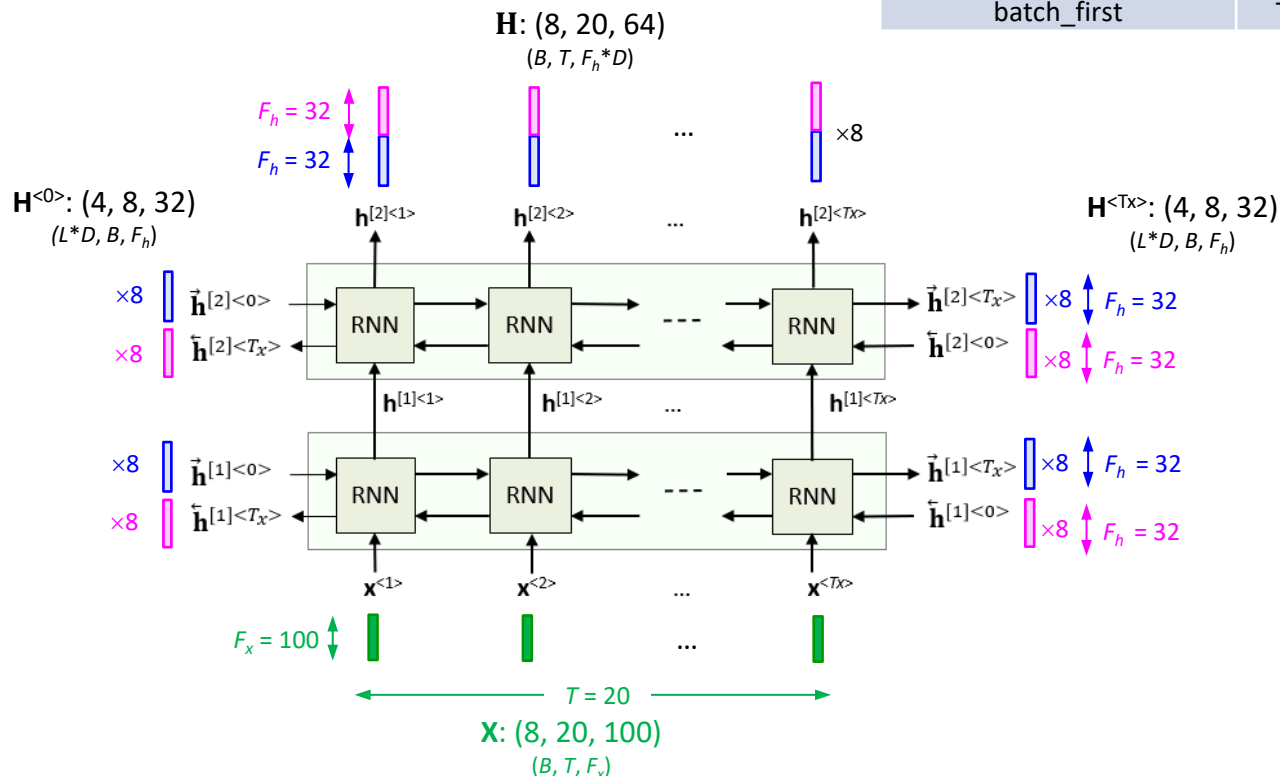
RNN Hyperparameters	Setting
num_layers, L	1
hidden_size, F_h	32
input_size, F_x	100
bidirectional, D	False
batch_first	False



Signal Dimensions in RNN

- Batch size $B = 8$, Sequence length $T = 20$

RNN Hyperparameters	Setting
num_layers, L	2
hidden_size, F_h	32
input_size, F_x	100
bidirectional, D	True
batch_first	True



*Thank you
for your attention*